📄 Invoice Management System – For Nafath Batch 9 Trainees

*Note: This task is only for trainees who have joined on 8th Feb 2026.

🎯 What You're Building

A simple invoice management system that can store up to **3 invoices** and perform basic operations on them. This system will help a small business track customer invoices.

📚 Understanding Your Data

Each invoice contains **4 pieces of information**:

1. **Invoice Number** (TEXT) - Example: "INV001"
2. **Customer Name** (TEXT) - Example: "Sara Ahmed"
3. **Amount** (NUMBER) - Example: 250.50
4. **Status** (TEXT) - Either "Paid" or "Unpaid"

Think of it like: You have 3 filing slots on your desk. Each slot can hold one invoice form with these 4 fields filled in.

🎮 Your System Must Have These Features

Main Menu with 5 Options:

1. Add Invoice
2. View All Invoices
3. Edit Invoice
4. Delete Invoice
5. Exit

The menu should keep showing repeatedly until the user chooses to exit.

📋 FEATURE 1: Add Invoice

What should happen:

- Check if there's space available (maximum 3 invoices)
- If space available:
    - Ask the user for invoice number

    - Ask for customer name
    - Ask for amount
    - Ask for status (Paid or Unpaid)
    - Store all this information
    - Increase your count of invoices
    - Show success message
- If no space available:
    - Show message "Cannot add more invoices! Maximum 3 reached."

Think about:

- Where will you store the first invoice? (What variable names?)
- Where will you store the second invoice? (Different variables?)
- How do you know which "slot" to fill next? (Need a counter!)
- How do you check if there's space available? (Compare counter to maximum)

Questions to ask yourself:

- What variables do I need to create before I can store invoices?
- How many variables total do I need for 3 invoices?
- How will I know whether to store in slot 1, 2, or 3?

📋 FEATURE 2: View All Invoices

What should happen:

- Check if any invoices exist
- If no invoices:
    - Display "No invoices stored yet!"
- If invoices exist:
    - Display a header like "=== ALL INVOICES ==="
    - Show each invoice in a clear format
    - Display: Invoice Number | Customer Name | Amount | Status
    - At the end, show total count of invoices

Think about:

- How do you check if any invoices exist? (Check your counter!)
- How do you show only the invoices that were actually added? (Don't show empty slots!)
- What's the best way to go through each invoice? (Loop? Multiple IF statements?)

Display format example:

=== ALL INVOICES ===

INV001 | Sara Ahmed | $250.50 | Paid

INV002 | Ali Hassan | $150.00 | Unpaid

INV003 | Fatima Omar | $300.00 | Paid

Total Invoices: 3

Questions to ask yourself:

- How do I loop through exactly the number of invoices that exist?
- For each position (1, 2, 3), how do I know which variables to display?
- Should I display anything special if the invoice is unpaid? (Maybe later!)

📋 FEATURE 3: Edit Invoice

What should happen:

- Check if any invoices exist to edit
- If no invoices:
  - Display "No invoices to edit!"
- If invoices exist:
  - Show the user which invoices are available (list them by position: 1, 2, 3)
  - Ask which invoice they want to edit (by position number)
  - Check if that position has an invoice
  - If valid position:
    - Show current values of that invoice (so user can see what's there)
    - Ask for NEW invoice number
    - Ask for NEW customer name
    - Ask for NEW amount
    - Ask for NEW status

        - Replace the old values with new values
        - Show success message
    - If invalid position:
        - Show "That invoice doesn't exist!"

Think about:

- How do you show which invoices are available? (Display their invoice numbers?)
- When the user picks position 2, how do you access invoice 2's data?
- How do you replace old values with new ones? (Read new input into the same variables!)
- Do you need to change the count when editing? (No! Same number of invoices)

Two approaches to consider:

Approach A: Edit All Fields at Once

- Ask for all 4 new values
- Replace all 4 pieces of information
- Simpler to understand

Approach B: Edit One Field at a Time

- Ask which field they want to change (number, customer, amount, or status)
- Only ask for that one new value
- Only replace that one piece
- More flexible but more complex

Choose whichever approach makes more sense to you!

Questions to ask yourself:

- How do I show the current values before asking for new ones?
- If the user wants to edit invoice 2, which variables do I need to update?
- What if they try to edit invoice 3 but only 2 invoices exist?

📋 FEATURE 4: Delete Invoice

What should happen:

- Check if any invoices exist to delete
- If no invoices:

    - Display "No invoices to delete!"
- If invoices exist:
    - Show which invoices are available
    - Ask which invoice to delete (by position number)
    - Check if that position exists
    - If valid position:
        - Remove that invoice
        - Handle what happens to the remaining invoices
        - Decrease your count
        - Show success message
    - If invalid position:
        - Show "That invoice doesn't exist!"

Think about - The Big Challenge!

When you delete an invoice, what happens to the others?

Imagine you have:

- Position 1: INV001
- Position 2: INV002
- Position 3: INV003

If you delete position 2, what should happen?

Two Different Approaches:

APPROACH A: Mark as Deleted (EASIER!)

Concept: Don't actually remove the invoice - just mark it as "deleted" so it doesn't show up anymore.

How it works:

- Add one extra piece of information to each invoice: IS_DELETED (TRUE or FALSE)
- Start all invoices with IS_DELETED = FALSE
- When deleting, just change IS_DELETED to TRUE for that invoice
- When viewing or counting, skip any invoice where IS_DELETED = TRUE

Advantages:

- Much simpler!
- No moving data around
- No complex logic

Disadvantages:

- The "deleted" data is still stored (just hidden)
- Your count might get confusing (do you count deleted ones or not?)

APPROACH B: Shift Invoices Up (HARDER!)

Concept: Actually remove the invoice and move the ones after it forward to fill the gap.

How it works:

Example - Deleting position 2:

1. Before: [Invoice1] [Invoice2] [Invoice3]
2. After: [Invoice1] [Invoice3] [Empty]

Steps:

- Take all the data from position 3
- Copy it into position 2
- Clear position 3
- Reduce count by 1

What if you delete position 1?

- Copy position 2 to position 1
- Copy position 3 to position 2
- Clear position 3
- Reduce count by 1

What if you delete position 3?

- Just clear position 3
- Reduce count by 1
- Nothing to shift!

Advantages:

- Actually removes the data

- Cleaner - no "ghost" invoices
- Count stays accurate

Disadvantages:

- Complex logic
- Lots of copying
- Easy to make mistakes

Choose Your Approach!

Recommendation for beginners: Use **Approach A** (Mark as Deleted)! It's much simpler and works just as well for this task.

Questions to ask yourself:

- Which approach makes more sense to me?
- If I use "mark as deleted", where do I check the IS_DELETED flag?
- If I use "shift up", what happens in each scenario (delete 1, delete 2, delete 3)?
- How does my count variable change after deletion?

📋 FEATURE 5: Exit Program

What should happen:

- When user chooses option 5
- Display a goodbye message like "Thank you for using Invoice Manager!"
- The menu should stop repeating
- Program should end

Think about:

- What loop condition makes the menu repeat?
- What value of CHOICE makes that condition become FALSE?
- Do you need to do anything special, or does the loop just stop naturally?

🎯 Additional Requirements

Invalid Input Handling:

Your program should handle these situations gracefully:

1. **Invalid menu choice:** If user enters 7 or 99 or -5
    - Display "Invalid choice! Please enter 1-5"
    - Show menu again
2. **Trying to add when full:** Already covered in Feature 1
3. **Trying to edit/delete non-existent invoice:**
    - If user tries to edit invoice 3 but only 2 exist
    - Display appropriate error message
4. **Trying to edit/delete when no invoices exist:** Already covered in Features 3 and 4

📐 Program Structure Guidance

Variables You Need to Think About:

For tracking:

- Total count of invoices stored
- User's menu choice
- User's edit choice (which invoice to edit)
- User's delete choice (which invoice to delete)

For storing invoice 1:

- Invoice 1's number
- Invoice 1's customer name
- Invoice 1's amount
- Invoice 1's status
- (Optional: Invoice 1's IS_DELETED flag)

For storing invoice 2:

- All the same types of variables as invoice 1, but with "2" in the names

For storing invoice 3:

- All the same types of variables as invoice 1, but with "3" in the names

Count them: How many variables total?

Program Flow:

Think about the overall structure:

1. **Setup Phase:**
    - Program starts
    - Create all your variables
    - Set initial values (counts to 0, deleted flags to FALSE, etc.)
2. **Main Loop Phase:**
    - Loop keeps running while user hasn't chosen exit
    - Each time through the loop:
        - Display menu
        - Read user choice
        - Check which option they chose
        - Execute that option's logic
        - Loop back to show menu again
3. **Ending Phase:**
    - User chose exit
    - Display goodbye message
    - Program stops

Decision Making Strategy:

For the main menu, you'll need to check which option the user chose:

- Is it option 1? Do add invoice logic
- Is it option 2? Do view invoices logic
- Is it option 3? Do edit invoice logic
- Is it option 4? Do delete invoice logic
- Is it option 5? Do exit logic
- Is it none of these? Show invalid choice message

Each option has its own set of sub-decisions inside it!

💡 Step-by-Step Building Strategy

Don't try to build everything at once! Build and test in phases:

Phase 1: The Foundation

- Create the program structure (START, BEGIN, END, STOP)
- Create the menu display
- Create the loop that keeps showing the menu
- Make exit (option 5) work
- **TEST:** Does menu show? Does it repeat? Can you exit?

Phase 2: Add One Invoice

- Create variables for just ONE invoice
- Create the counter variable
- Make option 1 work for adding ONLY the first invoice
- **TEST:** Can you add one invoice? Does count increase?

Phase 3: View One Invoice

- Make option 2 work to display the first invoice (if it exists)
- **TEST:** Add an invoice, then view it. Does it show correctly?

Phase 4: Support Three Invoices

- Create variables for second and third invoices
- Modify option 1 to decide which slot (1, 2, or 3) to fill
- Modify option 2 to show all invoices that exist
- **TEST:** Add 3 invoices. View them all. Does it work?

Phase 5: Add Edit Feature

- Implement option 3 completely
- **TEST:** Add some invoices, edit one, view to confirm changes

Phase 6: Add Delete Feature

- Implement option 4 using your chosen approach
- **TEST:** Add 3 invoices, delete middle one, view to see result

Phase 7: Polish

- Add all the error messages

- Test all the edge cases
- Make the display look nice

🤔 Problem-Solving Hints

When you get stuck, ask yourself:

1. **"What am I trying to accomplish right now?"**
    - Break it into the smallest possible step
2. **"What information do I have?"**
    - What variables exist?
    - What values are in them?
3. **"What information do I need?"**
    - What am I missing?
    - Where can I get it?
4. **"Have I done something similar before?"**
    - Can I copy and modify logic from another part?
5. **"Can I make this simpler?"**
    - Is there an easier way to think about this?

🎯 Concept Review - What You're Learning

This task teaches you:

✅ **Variables** - Storing different types of information
✅ **Input/Output** - Communicating with the user
✅ **Loops** - Repeating actions (menu loop, viewing multiple invoices)
✅ **Conditions** - Making decisions based on different situations
✅ **Counting** - Tracking how many items you have
✅ **Data Management** - Organizing related information
✅ **Validation** - Checking if inputs are valid
✅ **Problem Decomposition** - Breaking big problems into smaller pieces

---

✅ Success Criteria - How to Know You're Done

Your program is complete when:

- ✅ The menu displays clearly with all 5 options
- ✅ The menu repeats until the user chooses exit
- ✅ User can add up to 3 invoices successfully
- ✅ User cannot add more than 3 invoices (shows error message)
- ✅ User can view all invoices in a clear format
- ✅ Viewing shows "No invoices" when none exist
- ✅ User can edit any existing invoice
- ✅ Editing shows current values before asking for new ones
- ✅ User gets error message when trying to edit non-existent invoice
- ✅ User can delete any existing invoice
- ✅ After deletion, viewing invoices works correctly (doesn't show deleted ones)
- ✅ User gets error message when trying to delete non-existent invoice
- ✅ Invalid menu choices show an error message
- ✅ Program exits cleanly when option 5 is chosen

🎈 Final Encouragement

This is a challenging task, but you can do it!

Remember:

- Start small and build up gradually
- Test after every small change
- Don't move to the next feature until the current one works
- It's okay to use the simpler approaches (like "mark as deleted")
- Draw diagrams if it helps you visualize the data
- Write out examples on paper before coding

You're learning fundamental programming concepts that every programmer uses!

Take your time, be patient with yourself, and celebrate small victories! 🌟 Good luck! 💪